dbt Certification ��� Checkpoint 2: Govern and Debug Your Models

Checkpoint 2 — Govern and Debug Your Models (Meta Study Guide)

> Synthesized exam-prep guide for Checkpoint 2. Reorganizes the 7 per-resource cheat sheets by topic and shows how they fit together as a single story: **how to publish reliable model APIs and debug when things go wrong**.

The shape of Checkpoint 2

CP2 is significantly tighter than CP1 — 7 resources, two themes:

1. **Governance** — contracts, versions, constraints, data-product management. How to make a model into a trustworthy API for downstream consumers.
2. **Debug & control** — compile, debugging errors, behavior changes, global configs. How to inspect what dbt is doing and configure its behavior.

> ⚠️ The Checkpoint 2 readings list one item, *Data product management: best practices*, hosted at `www.getdbt.com/blog/data-product-management`. That post lives on dbt's marketing site, which is **not open-sourced** and was unreachable from this sandbox. I've flagged it here; if you can paste the contents, I can produce a per-resource cheat sheet for it. The synthesis below incorporates its themes based on the rest of the CP2 material and the Mesh courses it draws from.

1. The "models as APIs" mental model

Read: `01-model-contracts.md`, `02-model-versions.md`, `03-constraints.md`.

The core idea: a public dbt model has **producers** (your team) and **consumers** (other teams, dashboards, ML pipelines, other dbt projects). Producers want freedom to iterate; consumers want stability. CP2 is the toolkit that makes both possible:

Concern	Tool
What columns/types will the model produce?	**Contract**
How do I change those guarantees without breaking consumers?	**Versions** + **deprecation_date**
What guarantees does the warehouse enforce?	**Constraints**
How do producers and consumers coordinate at scale?	**Data product management practices** (the missing post)

This is conceptually the same arc you see for web APIs — define the contract, version it, deprecate old versions on a schedule, communicate clearly.

2. Contracts — shape guarantees enforced at build time

Read: `01-model-contracts.md`.

A contract defines a model's column names, data types, and (optionally) constraints. With `enforced: true`:

1. **Preflight check** — dbt verifies the SQL *would* produce columns matching the contract before sending DDL.
2. **DDL enforcement** — column types and constraints land in the `CREATE TABLE` statement; the warehouse enforces them on insert.

```yaml
models:
  - name: dim_customers
    config:
      materialized: table              # required: table or incremental, NOT view/ephemeral
      contract: {enforced: true}
    columns:
      - name: customer_id
        data_type: int
        constraints:
          - type: not_null
      - name: customer_name
        data_type: string
```

Hard rules

- Materialization must be `table` or `incremental`.
- **Every** column needs `data_type`.
- Ephemeral and view models can't carry contracts.

Contracts vs data tests

	Contract	Data test
Validates	**shape** (columns/types)	**content** (rows)
Failure effect	Build doesn't happen	Test reports failure
When	Build-time	After build
Configurable severity	No	Yes

For exam purposes: contracts catch *structural* breakage; tests catch *data quality* issues. Some platform-enforced constraints (`not_null` everywhere; everything else is platform-dependent) can replace equivalent data tests — cheaper, build-time enforcement.

Which models should have contracts?

Public models — anything other teams, projects (Mesh), or external systems consume. Internal intermediate models usually don't need them.

3. Versions — evolve models without breaking consumers

Read: `02-model-versions.md`.

When a contract change would break consumers (drop column, rename, type change), you don't just push it. You **create a new version**:

```yaml
models:
  - name: dim_customers
    latest_version: 1
    config:
      materialized: table
      contract: {enforced: true}
    columns:
      - name: customer_id
        data_type: int
      - name: country_name
        data_type: varchar
    versions:
      - v: 1                    # current latest; matches the shared spec

      - v: 2                    # the new shape
        columns:
          - include: all
            exclude: [country_name]   # breaking change: dropping country_name
```

How `ref` resolves

- `ref('dim_customers')` → resolves to **latest** automatically.
- `ref('dim_customers', v=1)` → pins to v1.
- dbt prints notices when an unpinned ref resolves to latest *and* a prerelease exists.

File / relation conventions

v	role	file name	relation
3	prerelease	`dim_customers_v3.sql`	`dim_customers_v3`
2	latest	`dim_customers.sql` or `dim_customers_v2.sql`	`dim_customers_v2` AND `dim_customers` (recommended)
1	old	`dim_customers_v1.sql`	`dim_customers_v1`

The deprecation loop

1. Develop new version.
2. Bump latest to point at it.
3. Mark old for deprecation (`deprecation_date` resource property).
4. Wait for consumers to migrate.
5. Delete old version.

Tactical notes

- Only version **public/contracted models** with downstream commitments. Don't version every change.
- Keep ≤ 2–3 live versions at a time (same rule as web APIs).
- Non-breaking additions (new column, bug fix) don't need a new version — update latest in place.
- Schedule periodic version bumps (1–2× per year) where you sweep out unused columns rather than versioning every small change.

Selecting by version

```yaml
selectors:
  - name: exclude_old_versions
    default: "{{ target.name == 'dev' }}"
    definition:
      method: fqn
      value: "*"
      exclude:
        - method: version
          value: old
```

Useful for skipping deprecated versions in dev while still building them in prod during migration.

4. Constraints — warehouse-enforced rules

Read: `03-constraints.md`.

Constraints attach to columns or models, declaring rules the warehouse enforces on insert. Types: `not_null`, `unique`, `primary_key`, `foreign_key`, `check`, `custom`.

Prerequisites

- Materialization: `table` or `incremental`.
- The model has an **enforced contract**.
- Every column declares `data_type`.

Column-level vs model-level

- ✅ Single-column constraints → **column-level**.
- ✅ Multi-column constraints (composite PK, multi-column FK, cross-column check) → **model-level**.
- Multiple `primary_key` constraints must be model-level.

```yaml
models:
  - name: dim_customers
    config: {materialized: table, contract: {enforced: true}}
    constraints:
      - type: primary_key
        columns: [customer_id, region]
      - type: foreign_key
        columns: [customer_id]
        to: ref('stg_customers')
        to_columns: [customer_id]
      - type: check
        columns: [start_date, end_date]
        expression: "start_date <= end_date"
        name: dim_customers_valid_window
    columns:
      - name: customer_id
        data_type: int
        constraints: [{type: not_null}]
```

Platform enforcement reality

- **Postgres** enforces everything.
- **Most analytical platforms** enforce only `not_null`; PK/UQ/FK are *informational*.
- Use `warn_unenforced: false` to silence noise on supported-but-not-enforced constraints; `warn_unsupported: false` for those the platform doesn't recognize.

Foreign keys (v1.9+)

```yaml
- type: foreign_key
  columns: [customer_id]
  to: ref('stg_customers')
  to_columns: [customer_id]
```

Using `ref()` for `to:` captures the dependency in the DAG and survives environment switches — much better than hardcoding schema names in `expression`.

Constraints vs data tests

Constraints win when the platform actually enforces them — they fail the build at insert time, no extra query. For everything informational, pair with a data test.

---

5. Data product management (the missing-blog gap)

The Checkpoint 2 readings list one resource I couldn't fetch — *Data product management: best practices* on `www.getdbt.com`. I'll note its themes here based on adjacent dbt material; replace this section if you can pull the actual post.

Why this complements the technical CP2 toolkit

Contracts, versions, and constraints give you the *mechanics* of treating models as APIs. The data-product-management practices give you the *organizational* side:

- **Producers** (data engineers / analytics engineers) are responsible for the contract, freshness SLA, and migration plan. They publish a versioned, documented product.
- **Consumers** (BI teams, ML engineers, downstream dbt projects via Mesh) are responsible for staying within the contract and migrating when notified.
- **The contract** is the formal boundary; everything else (tests, exposures, freshness, version-bump cadence) is the SLA.

This is the same idea as product management for software APIs: ownership, lifecycle, communication, deprecation.

How CP2 features support this division

Practice	Mechanism in dbt
"Declare the API"	Contract
"Communicate the API"	Description + doc blocks; rendered docs site
"Make consumers visible"	Exposures (CP3)
"Version & sunset"	Versions + `deprecation_date`
"Health monitoring"	Source freshness, data tests, data health tiles
"Restrict access"	Groups + `access: public/private/protected` (CP1's rest-of-project + Mesh)

Action items the missing post almost certainly recommends

- Treat each public model as a product with an owner, an SLA, and consumers.
- Bake the migration cadence (e.g., once-or-twice-yearly version bump) into the team's calendar.
- Use exposures and Catalog/Explorer to keep producers and consumers visible to each other.
- Tie test severity to consumer impact (this is also the *Test smarter* framework in CP3).

---

6. Debugging — the systematic approach

Read: `04-debugging-errors.md`.

The five-step loop

1. **Read the error message** — names the type and the file.
2. **Open the file** — fix is often obvious.
3. **Isolate** — run one model; revert recent changes.
4. **Use compiled files and logs**:
   - `target/compiled/<model>.sql` — bare `select`, runnable in a SQL client.
   - `target/run/<model>.sql` — the SQL dbt actually sent (with materialization DDL).
   - `logs/dbt.log` — every query dbt ran. Recent errors at the bottom.
5. **Ask for help** — write a good question, with the error + the file content.

Error type → likely cause

Type	Phase	Likely cause
`Runtime Error`	Initialize	Not a dbt project; bad profile; failed connection; invalid `dbt_project.yml`
`Compilation Error`	Parsing	Invalid Jinja; invalid YAML; bad `ref`; wrong schema key
`Dependency Error`	Graph validation	Circular `ref`; disabled model still ref'd
`Database Error`	SQL execution	Type mismatch; missing column; permissions; dialect mismatch

Tools that solve most debugging

- `dbt debug` — connection check.
- `dbt debug --config-dir` — find your `profiles.yml`.
- `dbt parse` — fast syntax check, no warehouse.
- `dbt compile` — render SQL; inspect what dbt is sending.
- `dbt show --select <test>` — see the failing rows for a test.
- `--debug` flag — verbose log output.
- `--fail-fast` — stop at first error.

Diagnostic pro tip

When a database error shows up in a downstream model but feels off, temporarily materialize the upstream chain as tables. The warehouse will throw the error at the real culprit.

---

7. `dbt compile`

Read: `05-cmd-compile.md`.

Renders compiled SQL into `target/` without executing it. Used to: - Inspect rendered Jinja/macros/refs. - Manually run a model's SQL in a SQL client for debugging. - Compile analyses (analyses aren't materialized any other way).

Common patterns

```
dbt compile                                    # everything
dbt compile --select stg_orders                # one model
dbt compile --inline "select * from {{ ref('raw_orders') }}"  # ad-hoc
dbt compile --no-introspect                    # avoid metadata queries

dbt --no-populate-cache compile                # skip warehouse cache warm-up
```

`compile` vs `parse` vs `run` vs `build`

Command	Compiles	Connects to warehouse	Executes
`dbt parse`	❌	❌	❌
`dbt compile`	✅	usually (cache/introspection)	❌
`dbt run`	✅	✅	✅ models
`dbt build`	✅	✅	✅ everything

v1.12+ extends `compile` to **snapshots** — you can finally inspect the generated SCD SQL.

---

8. Behavior changes — controlled migration of dbt's runtime

Read: `06-behavior-changes.md`.

A specific category of flags that gate **breaking dbt-runtime changes**. They live in `dbt_project.yml`'s `flags:` block, version-controlled and PR-reviewed.

Three-phase lifecycle

1. **Introduction** — new behavior available but disabled by default.
2. **Maturity** — default flips to `true`; old behavior still selectable but warned about.
3. **Removal** — flag and old behavior deleted from dbt.

What counts as a behavior change

- Same code + same command → different result.
- New errors (not warnings).
- Macro signature changes.
- Breaking changes to artifact JSON.

What's NOT a behavior change (handled in regular releases)

- Bug fixes.
- New warnings.
- New artifact fields with defaults.

The flags you'll see most often

```yaml
flags:
  require_explicit_package_overrides_for_builtin_materializations: true

  require_resource_names_without_spaces: true
  source_freshness_run_project_hooks: true
  state_modified_compare_more_unrendered_values:
false  # turn ON to reduce CI false positives
  validate_macro_args: false
  require_generic_test_arguments_property: true
```

Fusion erases the migration
All behavior-change flags are removed in Fusion — the new behavior is always on.

9. Global configs — how dbt runs

Read: `07-global-configs.md`.

Flags vs resource configs: - **Resource configs** describe **what** to build (materialization, tags, schema). - **Flags / global configs** describe **how** dbt runs (logging, fail-fast, threads, partial parse, defer, target).

Three layers of setting (most specific wins)

1. CLI option (`--full-refresh`).
2. Environment variable (`DBT_FOO` v1.10−; `DBT_ENGINE_FOO` v1.11+).
3. `dbt_project.yml` under `flags:`.

Common flags

- Logging: `debug`, `log_level`, `log_format`, `log_path`, `quiet`, `printer_width`, `use_colors`.
- Behavior: `fail_fast`, `warn_error`, `warn_error_options`, `partial_parse`, `populate_cache`.
- Targets: `target`, `profile`, `profiles_dir`, `project_dir`.
- Slim CI: `defer`, `defer_state`, `favor_state`, `state`.
- Build cost: `empty`, `sample`, `event_time_start`, `event_time_end`.
- Resource type: `resource-type`, `--exclude-resource-type`.
- Failures: `store_failures`.

Reading flags from Jinja

```
on-run-start:
  - '{{ log("I will fail fast", info=true) if flags.FAIL_FAST }}'
```

Don't use `flags` for parse-time-resolved configs (refs, sources).

10. Putting CP2 together

Imagine you own a public `dim_customers` mart consumed by three dashboards and another dbt project via Mesh.

1. **Contract.** Declare every column with its `data_type`. Add `not_null` and `primary_key` constraints on `customer_id`. Materialize as `table`.
2. **Tests.** PK uniqueness + not-null (data tests); business-anomaly tests on the calculated fields (CP3 *Test smarter*).
3. **Exposures.** Declare the three dashboards as exposures so the lineage is explicit (CP3).
4. **Version.** Today the model is v1. Tomorrow you need to drop `country_name`. Create `dim_customers_v2.sql`, declare `versions:` in YAML, set `latest_version: 2`, give v1 a `deprecation_date`. Inform consumers.
5. **Debug.** A consumer reports unexpected NULLs. You `dbt show --select unique_dim_customers_customer_id` to see failing rows; trace to the upstream staging model. Fix and `dbt build --select stg_customers+`.
6. **Behavior changes.** Upgrading to next dbt version surfaces a warning about `state:modified` comparisons. Turn on `state_modified_compare_more_unrendered_values` to reduce CI false positives.
7. **Compile.** Add a complex `check` constraint expression; `dbt compile --select dim_customers` to confirm the DDL renders correctly before running.

Exam-ready summary tables

Governance tool → role

Tool	Role
Contract	Shape guarantee (columns/types) enforced before/during build
Version	Mechanism for evolving contracts without breaking consumers
Constraints	Per-column / per-model rules the warehouse enforces
`deprecation_date`	Sunset signal for old versions
Exposures (CP3)	Make consumers visible

Materialization compatibility

	Contract	Constraints (enforced)
view	❌	❌
ephemeral	❌	❌
table	✅	✅
incremental	✅	✅
materialized_view	varies	varies

Behavior-change-flag lifecycle phases

1. Introduction (default `false`).
2. Maturity (default flips to `true`).
3. Removal (flag and old behavior deleted).
4. Fusion: all flags removed.

Compile-related commands

Command	When
`dbt parse`	Validate parse without warehouse
`dbt compile`	Render compiled SQL into `target/`
`dbt compile --inline`	One-off Jinja-SQL rendering
`dbt show`	Compile + execute + preview rows

File index (this directory)

- `01-model-contracts.md`
- `02-model-versions.md`
- `03-constraints.md`
- `04-debugging-errors.md`
- `05-cmd-compile.md`
- `06-behavior-changes.md`
- `07-global-configs.md`
- *(Missing — provided by user later if available)* `08-data-product-management.md`

How to study from here

1. Walk through the "publishing a public model" scenario in §10 mentally.
2. Memorize the contract prerequisites and the version naming conventions.
3. Know `not_null` is the only constraint most analytical warehouses enforce.
4. Reproduce the four error-type → likely-cause table.
5. Distinguish `flags:` from resource configs cold.
6. Move to Checkpoint 3.

Model Contracts

Source: https://docs.getdbt.com/docs/mesh/govern/model-contracts **Type:** Documentation · Checkpoint 2

What it is

A model contract is a set of upfront guarantees about a model's **shape** — its columns, their data types, and (optionally) constraints. With a contract enforced, dbt verifies before building that the model's SQL will produce a dataset matching those guarantees. If it doesn't, the build fails.

Why use one

A bare `select` model can change shape whenever its SQL changes. That's great for iteration, but bad for downstream consumers (dashboards, other dbt projects, external systems). Contracts protect those consumers by making the model's interface explicit and validated.

Defining a contract

```
# models/marts/customers.yml
models:
  - name: dim_customers
    config:
      contract:
        enforced: true
    columns:
      - name: customer_id
        data_type: int
        constraints:
          - type: not_null
      - name: customer_name
        data_type: string
```

When enforced: 1. **Preflight check** — dbt verifies the SQL will return columns with the declared names and `data_type`s. Column order doesn't matter at this stage. 2. **DDL inclusion** — dbt includes the column names, types, and constraints in the `CREATE TABLE` DDL it sends to the warehouse. The warehouse enforces them on insert. The DDL also orders columns to match the contract.

Requirements

- Every column must declare `name` AND `data_type`.
- Constraints (beyond just types) are **only** valid on `table` or `incremental` materializations — not on `view` or `ephemeral`.
- Constraints depend on platform support (see Constraints doc).

Contracts vs tests

	Model contract	Data test
Validates	**Shape** (columns/types)	**Content** (rows)
When	Build-time (preflight + DDL)	After build
Effect on build	Failed contract → no build	Failed test → table built, test reports
Configurable severity	No	Yes (`severity: warn` etc.)
Best for	"API" guarantee for downstream	Data quality

Some data tests can be replaced by **constraints** for build-time enforcement (cheaper, stricter): - Prerequisites: `table` or `incremental` materialization, full contract, platform actually enforces the constraint (most enforce only `not_null`). - Most platforms accept `primary_key`/`unique`/`check` as informational but don't enforce them at insert time.

Which models should have contracts?

Recommended for **public models** that other groups/teams/projects consume — internally via dbt Mesh or externally via dashboards/exposures. For private intermediate models, contracts are overkill.

All columns are required

A contract must cover **every** column the model selects. There's no partial-contract option. For wide models this means a lot of YAML — generation tools (`dbt-codegen`) help.

Breaking changes

When you change a contracted model in a way that violates the contract (drop a column, change a type), dbt detects it during state comparison: - **Versioned models** → error (breaking change requires bumping version). - **Unversioned models** → warning. - Removing a contracted model entirely (delete/rename/disable) → same rules apply (v1.9+).

This is how contracts pair with **model versions** — contracts make the breaking change detectable; versions give you a path through it.

Platform constraint support (high level)

- Postgres: full ANSI + `check` (all enforced).
- Most analytical platforms: enforce `not_null` only; others are informational.
- Use `warn_unenforced: false` and `warn_unsupported: false` per-constraint to silence warnings.

Key takeaways

- Contracts = shape guarantees enforced before and during build.
- `enforced: true` + `name` + `data_type` for every column.
- Constraints work only on `table` / `incremental` materializations.
- Distinct from tests: contracts validate shape, tests validate rows.
- Pair with **model versions** to manage breaking changes gracefully.
- Recommend for public/consumed models, not every internal one.

Related

- Constraints (the per-constraint reference).
- Model versions (how to evolve contracts safely).
- Exposures (mark downstream consumers).

Model Versions

Source: https://docs.getdbt.com/docs/mesh/govern/model-versions **Type:** Documentation · Checkpoint 2

What it is

A mechanism for shipping multiple **live, simultaneous versions** of the same model so producers can iterate on breaking changes while consumers get a migration window. Versioning is to dbt models what semver is to public APIs.

The producer/consumer tension

- **Producer** wants to evolve logic and structure.
- **Consumer** wants stability and notice before things break.

Model versions resolve this by letting both happen concurrently for a while.

When to version

- A change is **breaking**: column dropped, renamed, type changed, nullability flipped — anything that breaks a contracted shape.
- A recalc that doesn't break the contract but materially changes results (judgment call).

Don't version every change. Non-breaking additions (new column, bug fix in an existing column's logic) should go on the latest version. Plan periodic version bumps (1–2× a year) where you sweep out unused columns.

Versions vs version control vs new model

	Version control	New model	Model versions
Lives in repo?	Always 1 state on `main`	Independent model	Multiple live versions
Deployed at once?	One environment at a time	One name	Multiple at once
Notifies consumers?	No	No	Yes (prerelease/ deprecation)
Reusable config?	–	No	Yes (define diffs only)

How `ref` resolves

- `ref('dim_customers')` → resolves to the model's **latest** version automatically.
- `ref('dim_customers', v=1)` → pins to v1.

dbt prints helpful messages when an unpinned `ref` resolves to latest and a prerelease is available:

```
Found an unpinned reference to versioned model 'dim_customers'.
Resolving to latest version: my_model.v2
A prerelease version 3 is available...
```

File / relation naming convention

v	version	`ref` syntax	File name	Relation
3	prerelease	`ref('dim_customers', v=3)`	`dim_customers_v3.sql`	`analytics.dim_customers_v3`
2	latest	`ref('dim_customers')` or `ref(..., v=2)`	`dim_customers_v2.sql` or `dim_customers.sql`	`analytics.dim_customers_v2` **and** `analytics.dim_customers` (recommended)
1	old	`ref('dim_customers', v=1)`	`dim_customers_v1.sql`	`analytics.dim_customers_v1`

Defining versions in YAML

Diffs-only style (recommended)

```yaml
models:
  - name: dim_customers
    latest_version: 1
    config:
      materialized: table
      contract: {enforced: true}
    columns:
      - name: customer_id
        data_type: int
      - name: country_name
        data_type: varchar
    versions:
      - v: 1
        # matches what's above
      - v: 2
        # breaking change: dropped country_name
        columns:
          - include: all
            exclude: [country_name]
```

Fully specified

You can also re-state config and columns inline under each version entry.

`include` / `exclude` helpers

`include: all` plus `exclude: [col1, col2]` produces the diff cleanly — useful when wide models change only a few columns.

Deprecation

Pair versions with `deprecation_date` to announce a sunset date for old versions. dbt warns consumers as the date approaches. Once past it, the version should be deleted from the project.

The whole loop: **develop new version** → **bump latest** → **mark old for deprecation** → **wait for migration** → **delete old**.

Selection by version

`dbt-` selectors support `method: version`:

```yaml
selectors:
  - name: exclude_old_versions
    default: "{{ target.name == 'dev' }}"
    definition:
      method: fqn
      value: "*"
      exclude:
        - method: version
          value: old
```

Lets you skip old versions in dev while still building them in prod during the migration window.

Practical guidance

- Version only models with downstream commitments (public/contracted models).
- Avoid having more than 2–3 live versions at a time — same wisdom as web APIs.
- Communicate deprecation dates clearly and well in advance.
- Each version has its own warehouse relation — be aware of storage cost.

Key takeaways

- Model versions = multiple live versions of one logical model.
- Breaking changes → new version; non-breaking → just update latest.
- `ref(name, v=N)` to pin; `ref(name)` for latest.
- Define in YAML with `versions:` + `latest_version` + per-version diffs.
- Pair with `contract` to detect breaks and `deprecation_date` to sunset.

Related

- Model contracts (catches the breaking change).
- `deprecation_date` resource property.

- `ref` with `version` argument.
- `state:modified.body` / version-aware state selection.

Constraints

Source: https://docs.getdbt.com/reference/resource-properties/constraints **Type:** Documentation · Checkpoint 2

What they are

Platform-enforced data integrity rules attached to a model's columns or the model itself. When supported and enforced by the warehouse, they reject invalid data at insert/load time — failing the build cleanly rather than silently writing bad rows.

Prerequisites

- Materialization is `table` or `incremental` — **never** `view` or `ephemeral`.
- The model has an enforced **contract** (`contract: {enforced: true}`).
- Every column has `data_type` declared (consequence of the contract requirement).

Constraint types

`not_null`, `unique`, `primary_key`, `foreign_key`, `check`, `custom`.

Structure

```
        constraints:
          - type: <type>             # required
            expression: "..."         # required for check; optional otherwise
            name: "..."               # optional, platform-dependent
            columns: [c1,
c2]         # model-level only — list of cols the constraint covers
            # v1.9+ foreign-key inputs
            to: ref('other_model')    # or source('src','tbl')
            to_columns: [other_col]
            # warning controls
            warn_unenforced: false    # supported but not enforced (e.g. PK on
Snowflake)
            warn_unsupported: false   # not supported by platform (e.g. check on
Redshift)
```

Column-level vs model-level

- ✅ **Column-level**: prefer this for single-column constraints.
- ✅ **Model-level**: required for *multi-column* constraints (e.g., composite primary keys, multi-column FKs, cross-column `check`).
- **Multiple `primary_key`** constraints must be defined at the model level. Column-level multi-PK is **not** supported.

```
        models:
          - name: dim_customers
            config:
              materialized: table
              contract: {enforced: true}
            constraints:
              - type: primary_key
                columns: [customer_id, region]
              - type: foreign_key
                columns: [customer_id]
                to: ref('stg_customers')
                to_columns: [customer_id]
              - type: check
                columns: [start_date, end_date]
                expression: "start_date <= end_date"
                name: dim_customers_valid_window
            columns:
              - name: customer_id
                data_type: int
                constraints:
                  - type: not_null
              - name: status
                data_type: varchar
                constraints:
                  - type: check
                    expression: "status in ('active', 'inactive')"
```

Foreign keys (v1.9+)

The `to` + `to_columns` form uses `ref()`/`source()` — that means: - dbt captures the dependency in the DAG. - The reference works across dev/CI/prod (no hardcoded schemas).

Pre-1.9 / unsupported adapters: fall back to a free-text `expression`.

Platform support overview

Platform	`not_null`	`unique`/`PK`	`FK`	`check`
Postgres	enforced	enforced	enforced	enforced
Redshift	enforced	informational	informational	not supported
Snowflake	enforced	informational	informational	informational
BigQuery	enforced	informational	informational	not supported
Databricks/Spark	enforced	informational	informational	varies

(See Model contracts doc for the full platform matrix.)

What "unenforced" means

The constraint is included in DDL metadata so external tools (catalogs, ER diagram tools) see it — but the warehouse will happily insert violating rows. For these, pair with a `unique`/`not_null` **data test** to actually catch bad data.

Constraints vs data tests

- **Constraints** validate at insert time. Cheap (no extra query) but limited (most platforms enforce only `not_null`).
- **Data tests** validate after build. Flexible, configurable severity, can run anywhere — and you can `store_failures` for inspection.

Use constraints where they're **enforced** and you need build-time stops. Use data tests where you need flexibility or the platform doesn't enforce.

Postgres example DDL produced

```
        create table "db"."schema"."constraints_example__dbt_tmp" (
            id integer not null primary key check (id > 0),
            customer_name text,
            first_transaction_date date
        );
```

Common patterns

```
        # Composite PK
        constraints:
          - type: primary_key
            columns: [order_id, line_item_id]

        # FK to another model
        - type: foreign_key
          columns: [customer_id]
          to: ref('dim_customers')
          to_columns: [customer_id]

        # Cross-column check
        - type: check
          expression: "shipped_at >= ordered_at"
          columns: [shipped_at, ordered_at]

        # Silence warnings on unenforced
        - type: primary_key
          columns: [id]
          warn_unenforced: false
```

Key takeaways

- Constraints require an enforced contract on a `table` or `incremental` model.
- Six types: `not_null`, `unique`, `primary_key`, `foreign_key`, `check`, `custom`.
- Column-level for single columns; model-level for multi-column or multiple PKs.
- Most analytical warehouses only enforce `not_null` — others are informational.
- Use `warn_unenforced`/`warn_unsupported` to silence noise.
- Pair informational constraints with data tests to actually catch violations.

Related

- Model contracts (precondition).
- Data tests, custom generic tests, `store_failures`.
- Adapter resource configs for per-platform details.

Debugging Errors

Source: https://docs.getdbt.com/guides/debug-errors **Type:** Documentation · Checkpoint 2

The general debugging loop

1. **Read the error message.** dbt error messages name the error *type* and the *file* where it happened.
2. **Open the offending file.** Often the fix is obvious.
3. **Isolate.** Run one model at a time; revert the change that broke things.
4. **Use compiled files and logs.**
   - `target/compiled/` — `select` statements you can run in any SQL editor.
   - `target/run/` — the SQL dbt executes (with materialization wrappers).
   - `logs/dbt.log` — all queries dbt ran, plus structured logging. **Recent errors at the bottom.**
   - dbt Cloud: use the `Details` tab in command output.
5. **Ask for help** — but write a good question first.

Error types (by execution phase)

Phase	What dbt is doing	Error type
Initialize	Confirm this is a dbt project + can reach the warehouse	`Runtime Error`
Parsing	Validate Jinja in `.sql`, YAML in `.yml`	`Compilation Error`
Graph validation	Build dependency graph, check it's acyclic	`Dependency Error`
SQL execution	Run the models	`Database Error`

Runtime Errors (initialization)

“Not a dbt project”

Missing `dbt_project.yml` in CWD or parents. - `pwd` to check directory. - `ls` to confirm `dbt_project.yml` is there.

“Could not find profile”

The `profile:` key in `dbt_project.yml` doesn’t match any block in `profiles.yml`. Usually a typo (singular vs plural). Run `dbt debug --config-dir` to locate `profiles.yml`.

“Failed to connect”

Credentials wrong. Update `profiles.yml`, then `dbt debug` to test:

```
dbt debug
# Connection test: OK connection ok
```

Invalid `dbt_project.yml`

“Additional properties are not allowed (‘hello’ was unexpected)” — you have a key dbt doesn’t recognize. Either remove/rename it, or check `dbt --version` (key may belong to a newer release).

Compilation Errors (parsing)

Invalid `ref`

“depends on a node named ‘stg_customer’ which was not found” — typo on a model name. Open the SQL file, fix the `ref()` argument, or rename the upstream file.

Invalid Jinja

“Reached EOF without finding a close tag” — missing `{% endmacro %}`, `{% endif %}`, `}`, etc. Read the line number in the error.

Invalid YAML

Indentation error → the YAML parser can’t build a dict. Use: - Indentation guides in your editor. - A YAML validator (yamllint, yamllint.com).

Incorrect YAML spec

YAML is well-formed but uses a key dbt doesn’t recognize for that resource. Check the reference for the resource type.

Dependency Errors (graph)

- **Circular dependency** — Model A `refs` Model B which `refs` Model A. Break the cycle (often by pulling a shared concept up into an intermediate).
- **Disabled model still referenced** — A disabled model can’t be `ref`’d. Either enable it or remove the reference.

Database Errors (execution)

The warehouse rejected the SQL dbt sent. - Read the warehouse-specific error in the dbt error message. - Open `target/run/<model>.sql` (the exact SQL dbt sent) and try it in your warehouse client. - Common causes: type mismatch, missing column, permissions, syntax dialect difference.

Useful commands for debugging

Command	What it tells you
`dbt debug`	Project setup + connection test
`dbt debug --config-dir`	Where your `profiles.yml` lives
`dbt parse`	Parse without executing — finds syntax errors fast
`dbt compile --select my_model`	Renders the SQL to `target/compiled/`
`dbt compile --inline "..."`	Compile arbitrary Jinja-SQL inline
`dbt show --select unique_my_model_id`	Preview a failing test’s results
`dbt run --select my_model --debug`	Verbose log output
`dbt run --fail-fast`	Stop at first error

Pro tips

- Always compile before running on big projects — it’s cheap and catches Jinja errors.
- For models with stacked views/ephemerals, an SQL error can appear in a downstream model that’s “really” caused upstream. Temporarily materialize the chain as `table` to surface the error where it originated.
- Keep `dbt.log` open in a separate terminal pane during debugging — `tail -f`.
- `dbt show` is the fastest way to see what a failing test query actually returns.

Key takeaways

- Four error types map to four phases of execution — error type tells you where to look.
- `target/compiled/` and `target/run/` are your friends — the SQL dbt actually generated.
- `dbt.log`, `--debug`, and `dbt debug` cover most setup issues.
- For database errors, copy the SQL out of `target/run/` and run it manually.
- `dbt show` lets you peek at the rows behind any test or model.

Related

- `dbt compile` (inspect generated SQL).
- `dbt show` (preview results).
- `dbt parse` (fast parse-only check).
- Global configs / `--debug`, `--fail-fast`.

`dbt compile`

Source: https://docs.getdbt.com/reference/commands/compile **Type:** Documentation · Checkpoint 2

What it does

Generates executable SQL for models, data tests, analyses, functions (and, in v1.12+, snapshots) **without executing it against the warehouse**. The output lands in `target/`.

What you can compile

- Models (v1.7+)
- Data tests
- Analyses
- Functions (UDFs)
- Snapshots (v1.12+)

Why use it

- **Inspect compiled SQL** — verify Jinja, macros, and refs resolved correctly.
- **Manually run a model’s SQL** — copy the compiled `select` and execute in your warehouse client to debug a database error.
- **Render analyses** — the only way to materialize their SQL (analyses aren’t built by `dbt run`).

Common misconceptions

- ❌ `dbt compile` is NOT required before `dbt run`/`dbt build`/`dbt test`. Those commands compile internally.
- ❌ Use `dbt parse` (not `compile`) if you only want to validate that the project loads, no warehouse connection involved.

Interactive compile (v1.5+)

Print compiled SQL directly to the terminal — useful for one-offs.

`--select` a single named node

```
dbt compile --select stg_orders
```

Output:

```
Compiled node 'stg_orders' is:
with source as (
    select * from "jaffle_shop"."main"."raw_orders"
), renamed as (
    select id as order_id, user_id as customer_id, order_date, status
    from source
)
select * from renamed
```

`--inline` an arbitrary dbt-SQL query

```
dbt compile --inline "select * from {{ ref('raw_orders') }}"
```

Output:

```
Compiled inline node is:
select * from "jaffle_shop"."main"."raw_orders"
```

Both forms also write to `target/` in addition to printing.

Performance/connection flags

`--no-populate-cache` (dbt-level flag)

Skips initial cache population of warehouse metadata. If metadata is later needed, dbt incurs a cache miss and queries the warehouse on demand.

```
dbt --no-populate-cache compile --select my_model
```

`--no-introspect` (compile-level flag)

Disables introspective queries. If a resource definition requires running one (e.g., to expand `*` to column names), dbt raises an error rather than connecting.

```
dbt compile --no-introspect
```

⚠️ When models use introspective queries (`run_query`, `get_columns_in_relation`, etc.), compiled SQL depends on warehouse metadata. Compilation may be incomplete or inconsistent depending on the warehouse state at compile time.

Where the SQL lives

Folder	Contains
`target/compiled/<project>/<path>/<model>.sql`	The bare compiled `select`
`target/run/<project>/<path>/<model>.sql`	The compiled SQL wrapped in materialization DDL (`create table as`, `merge into`, etc.)

Use the **compiled** version for ad-hoc query/debug; the **run** version to see exactly what dbt would send.

Common patterns

```
        # Compile everything
        dbt compile

        # Just one model
        dbt compile --select stg_orders

        # A test
        dbt compile --select unique_stg_orders_order_id

        # Ad-hoc Jinja
        dbt compile --inline "select count(*) from {{ ref('stg_orders') }} where
status = 'returned'"

        # Compile without hitting the warehouse for metadata
        dbt --no-populate-cache compile --no-introspect

        # Compile snapshots (v1.12+) — generates SQL with dbt_valid_from/to logic

        dbt compile --select my_snapshot
```

Compile vs run vs build vs parse

Command	Compiles SQL	Connects to warehouse	Executes SQL
`dbt parse`	❌ (syntax check only)	❌	❌
`dbt compile`	✅	usually (for cache/introspection)	❌
`dbt run`	✅	✅	✅ (models only)
`dbt build`	✅	✅	✅ (everything)

Key takeaways

- `dbt compile` renders compiled SQL into `target/` without executing.
- Two ways to inspect in-terminal: `--select <node>` and `--inline "<sql>"`.
- Not a prerequisite for `run`/`build`/`test` — they compile internally.
- For pure project parsing without warehouse access, use `dbt parse`.
- For metadata-free compile, combine `--no-populate-cache` and `--no-introspect`.
- v1.12+ extends compile to snapshots.

Related

- `dbt parse` — parse-only validation.
- `dbt show` — compile + execute + preview rows.
- Debugging errors guide — the consumer of compile output.

Behavior Changes (Flags)

Source: https://docs.getdbt.com/reference/global-configs/behavior-changes **Type:** Documentation · Checkpoint 2

What they are

A specific category of dbt flags that **opt projects into new runtime behaviors gradually**. They give a migration window between an old behavior and a new, better one. Set in the `flags:` dictionary in `dbt_project.yml`.

In **dbt Fusion**, all behavior-change flags are removed — the new behavior is always on. Read this doc primarily for Core 1.x and dbt Latest release track migrations.

Why they exist

- **Better defaults for new users** without breaking existing projects overnight.
- **Migration window for existing projects** — opt in when ready, see warnings until you do.
- **Maintainability for dbt itself** — every branching behavior is testing/cognitive overhead; flags are meant to be temporary.

The lifecycle (three phases)

1. **Introduction** — new behavior gated behind flag, **disabled by default**. Old behavior preserved.
2. **Maturity** — default flips to `true`. New behavior is the default. You can still opt out (`false`) to keep old behavior, but you'll see deprecation warnings.
3. **Removal** — flag and old behavior deleted from dbt entirely. Significant advance notice given.

What counts as a "behavior change"

Same project code + same command → different result.

Examples: - dbt newly raises a validation **error** (not a warning). - Built-in macro signature changes (your custom override may break). - Adapter renames/removes a method on `{{ adapter }}`. - Breaking change to artifact JSON contract (removed required field, etc.). - Removed structured-log field.

Not behavior changes: - Bug fixes (the prior behavior was defective). - New **warnings** (not errors). - Updated human-readable log strings. - Non-breaking artifact additions.

Where to set them

`dbt_project.yml`, under `flags:`. They're project-code-adjacent, so they belong in version control with PR review.

```
        flags:
          require_explicit_package_overrides_for_builtin_materializations: true

          require_resource_names_without_spaces: true
          source_freshness_run_project_hooks: true
          skip_nodes_if_on_run_start_fails: false
          state_modified_compare_more_unrendered_values: false
          validate_macro_args: false
          require_generic_test_arguments_property: true
          # ... many more
```

To opt out of a matured flag (keep old behavior): set its value to `false`. You'll see warnings — either fix the underlying issue (flag → `true`) or suppress via `warn_error_options.silence`.

Examples of common behavior-change flags

Flag	Effect when `true`
`require_explicit_package_overrides_for_builtin_materializations`	Packages can't silently override `table`/`view`/`incremental` etc.
`require_resource_names_without_spaces`	Spaces in model names raise an error
`source_freshness_run_project_hooks`	`dbt source freshness` runs `on-run-start`/`-end` hooks
`skip_nodes_if_on_run_start_fails`	Failed `on-run-start` hook skips selected resources
`state_modified_compare_more_unrendered_values`	Compare unrendered values during `state:modified` — reduces false positives across envs
`require_yaml_configuration_for_mf_time_spines`	MetricFlow time spine must be in YAML, not a SQL config block
`require_batched_execution_for_custom_microbatch_strategy`	Custom microbatch macros run in batches
`validate_macro_args`	dbt validates macro call arguments against declarations
`require_generic_test_arguments_property`	Generic tests must use `arguments:` instead of inline kwargs
`enable_truthy_nulls_equals_macro`	Null-safe equality macro behavior

Adapter-specific flags

Adapters (Databricks, Redshift, BigQuery, Snowflake) ship their own behavior-change flags too — for things like `use_info_schema_for_columns` (Databricks), `redshift_skip_autocommit_transaction_statements`, `bigquery_use_batch_source_freshness`,

`snowflake_default_transient_dynamic_tables`. Same lifecycle pattern; same `flags:` block.

Practical migration playbook

1. After upgrading, look for deprecation warnings in your run logs.
2. For each flag with TBD maturity: assess whether you can adopt now.
3. To adopt: fix the underlying issue, set the flag to `true`, confirm builds pass.
4. To defer: set explicitly to `false` so the warning doesn't surprise you when the default flips later.
5. Track deprecation/email notifications — they're sent before maturity dates.

Key takeaways

- Behavior-change flags = controlled migration mechanism for breaking changes.
- Three phases: introduction → maturity → removal.
- Configure in `flags:` block of `dbt_project.yml`.
- Default `true` = the new behavior is on; set `false` to revert (you'll see warnings).
- **Fusion removes them entirely** — new behavior always wins.

Related

- Deprecations index (`/reference/deprecations`).
- About global configs (the broader flag system).
- `warn_error_options` (silencing matured-but-opted-out warnings).
- Upgrading guides per dbt release.

About Flags (Global Configs)

Source: https://docs.getdbt.com/reference/global-configs/about-global-configs **Type:** Documentation · Checkpoint 2

What they are

"Flags" (a.k.a. global configs) control **how** dbt runs — log levels, fail-fast behavior, partial parsing, color output, caching, defer, threads, target, etc. They differ from **resource configs** (which describe **what** to build).

Where you can set them

Three layers. Most flags can be set in more than one; CLI > env var > project.

1. **CLI option** at invocation time (`--full-refresh`, `--debug`).
2. **Environment variable** — `DBT_FOO` (v1.10 and earlier) or `DBT_ENGINE_FOO` (v1.11+).
3. **`dbt_project.yml` `flags:` block**.

```yaml
# dbt_project.yml
flags:
  fail_fast: true
  partial_parse: true
  use_colors: true
  warn_error_options:
    include: all
    silence: ["NoNodesForSelectionCriteria"]
```

Some flags can **only** live in `dbt_project.yml`. Some can **only** be set per-invocation. The reference table in this doc lists each flag's allowed locations.

Accessing flags from Jinja

The `flags` context variable exposes flag values inside templates:

```yaml
on-run-start:
  - '{{ log("I will stop at the first sign of trouble", info=true) if flags.FAIL_FAST }}'
```

⚠️ Don't use `flags` to drive `ref`/`source`/configurations that dbt resolves at parse time — flags can vary per invocation, parse-time resolution can't.

Commonly used flags (quick reference)

Flag	Type	What it does
`target` (`--target dev`)	string	Which target/profile to use
`profile`	string	Which top-level profile in `profiles.yml`
`debug` (`--debug`)	bool	Verbose logging
`fail_fast` (`-x`)	bool	Stop on first error
`full_refresh` (`--full-refresh`)	bool	Force-rebuild incremental models
`empty` (`--empty`)	bool	Build schemas with no rows
`sample` (`--sample`)	string	Time-bounded sample data
`partial_parse`	bool (default true)	Reuse parser state across runs for speed
`populate_cache`	bool (default true)	Warm warehouse metadata cache
`introspect`	bool (default true)	Allow introspective queries during compile
`defer` / `state` / `favor_state`	bool/path	Slim CI defer pattern
`warn_error` / `warn_error_options`	bool/dict	Treat warnings as errors
`store_failures`	bool	Persist failing test rows to a table
`printer_width`	int	Console width for output
`use_colors`	bool	Colored output
`write_json`	bool	Write `manifest.json` etc.
`log_level` / `log_format` / `log_path`	enum/path	Log shaping
`quiet`	bool	Suppress non-error logs
`resource-type` / `--exclude-resource-type`	string	Filter what runs
`event_time_start` / `event_time_end`	datetime	Microbatch backfill window
`cache_selected_only`	bool	Cache only nodes you selected

CLI vs project — when each makes sense

- **CLI**: per-invocation overrides — `--target prod`, `--full-refresh`, `-x` to fail fast.
- **Env var**: CI/CD secrets (`DBT_PROFILES_DIR`) and consistent invocation context (`DBT_TARGET=prod`).
- **`dbt_project.yml`**: durable defaults the team needs every time (`partial_parse: true`, `printer_width: 120`).

Targets (`--target`)

Run the same code against different environments:

```
dbt run --target dev
dbt run --target prod
dbt build --target staging
```

Targets live in `profiles.yml`. Without `--target`, dbt uses the profile's default target.

The v1.11+ env-var rename

Starting v1.11, env vars are prefixed `DBT_ENGINE_` instead of `DBT_` (e.g., `DBT_ENGINE_FAIL_FAST` vs `DBT_FAIL_FAST`). Functional behavior is identical. v1.10 and earlier still use `DBT_`.

Behavior change flags (related but distinct)

There's a separate category of flags called **behavior changes** that gate migration between old and new dbt behaviors. They have a strict lifecycle (intro → maturity → removal) and live in the same `flags:` block. See *Behavior changes* cheat sheet.

High-impact recipes

```
# Slim CI
dbt build --select state:modified+ --defer --state path/to/prod

# Diagnose a flaky build with full logs
dbt --debug build --log-level debug --fail-fast

# CI-friendly: warnings become errors
dbt build --warn-error

# Zero-cost compile validation
dbt build --empty

# Store failing test rows for inspection
dbt test --store-failures
```

Key takeaways

- Flags = how dbt runs; resource configs = what dbt builds.
- Set in three places: CLI, env var, `dbt_project.yml`. CLI overrides env overrides project.
- Use `flags` Jinja var to read values in macros/hooks (but not at parse-time-resolved configs).
- Each flag's reference page lists where it can be set, env-var name, and CLI form.
- v1.11+ rename: `DBT_FOO` → `DBT_ENGINE_FOO`.

Related

- Behavior changes (cousin doc — migration-controlling flags).
- Environment variable configs.
- Slim CI pattern (uses defer/state/favor_state).
- `warn_error_options` (granular warning-to-error control).